Personal Project

Single-Cycle CPU

RV32I

Verilog

Yacine Ouhrouche

Started:

13/02/26

Finished:

28/02/26

Table of Contents

1.Project Overview 3

2.Development Environment 3

3. CPU Architecture 3

4. RV32I Instruction Format 4

5. Control Signal 8

6. Hardware Modules 9

7. Datapath 11

8. Simulations and Results 14

9. Limitation of Design 18

10.Summary 18

11. Future Work 19

1. Project Overview

This project consists of the design and implementation of a 32-bit single-cycle RISC-V processor supports a subset of the RV32I base integer instruction set. The processor is implemented using Verilog HDL and follows Harvard architecture to eliminate structural hazards and simplify memory access in a single-cycle implementation. A hardwired control unit was chosen to reduce latency and simplify combinational decoding logic. A subset of the RV32I base instruction set was selected to maintain compliance with the official RISC-V specification while keeping the design manageable.

2. Development Environment

The project is developed and verified using the following tools:

- **VSCode:** Used as the dev environment for writing and organizing Verilog source files.
- **Icarus Verilog**: Used for compilation and simulation of the Verilog source files.
- **Surfer:** Used to visualize simulation waveform and verify correct CPU behavior.

3. CPU Architecture Specifications

- Architecture: RV32I
- Data Width: 32-bit
- Instruction Width: 32-bit
- Register Count: 32 (x0–x31)
- Register x0: Hardwired to zero
- Memory Architecture: Harvard
- Execution Model: Single-cycle

4. RV32I Instruction Format

4.1 R-Type Format

31	25	24	20	19	15	14	12	11	7	6	0
Func 7		Rs2		Rs1		Func3		rd		Opcode	

R-Type Instructions

Name	Func7	Func3	Descr	Opcode
ADD	0000000	000	rd = rs1 + rs2	0110011
SUB	0100000	000	rd = rs1 - rs2	0110011
XOR	0000000	100	rd = rs1 ^ rs2	0110011
OR	0000000	110	rd = rs1 | rs2	0110011
AND	0000000	111	rd = rs1 & rs2	0110011
SLL	0000000	001	rd = rs1 << rs2	0110011
SRL	0000000	101	rd = rs1 >> rs2	0110011
SRA	0100000	101	rd = rs1 >> rs2	0110011
SLT	0000000	010	d = (rs1 < rs2) ? 1 : 0	0110011
SLTU	0000000	011	rd = (rs1 < rs2) ? 1 : 0	0110011

4.2 I-Type Format

31	20	19	15	14	12	11	7	6	0
imm		Rs1		Func3		rd		Opcode	

I-Type Instructions

Name	Func7	Func3	Descr	Opcode
ADDI	-	000	rd = rs1 + imm	0010011
XORI	-	010	rd = rs1 ^ imm	0010011
ORI	-	110	rd = rs1 | imm	0010011
ANDI	-	111	rd = rs1 & im	0010011
SLLI	0000000	001	rd = rs1 << imm	0010011
SRLI	0000000	101	rd = rs1 >> imm	0010011
SRAI	0100000	101	rd = rs1 >> imm	0010011
SLTI	-	010	d = (rs1 < imm) ? 1 : 0	0010011
SLTUI	-	011	rd = (rs1 < imm) ? 1 : 0	0010011

I-Type Loads

Name	Func7	Func3	Descr	Opcode
LB	-	000	rd = M[rs1+imm][0:7]	0000011
LH	-	001	rd = M[rs1+imm][0:15]	0000011
LW	-	010	rd = M[rs1+imm][0:31]	0000011
LBU	-	100	rd = M[rs1+imm][0:7]	0000011
LHU	-	101	rd = M[rs1+imm][0:15]	0000011

4.3 S-Type Format

31	20	19	15	14	12	11	7	6	0
Imm[11:5]		Rs1		Func3		Imm[4:0]		Opcode	

S-Type Instructions

Name	Func7	Func3	Descr	Opcode
SB	-	000	M[rs1+imm][0:7] = rs2[0:7]	0100011
SH	-	001	M[rs1+imm][0:15] = rs2[0:15]	0100011
SW	-	010	M[rs1+imm][0:31] = rs2[0:31]	0100011

4.4 B-Type Format

31	25	24	20	19	15	14	12	11	7	6	0
Imm[12|10:5]		Rs2		Rs1		Func3		Imm[4:1|11]		Opcode	

B type Instruction

Name	Func7	Func3	Descr	Opcode
BEQ	-	000	If(rs1==rs2) pc+=im	1100011
BNE	-	001	If(rs1!=rs2) pc+=im	1100011
BLT	-	100	If(rs1<rs2) pc+=im	1100011
BGE	-	101	If(rs1>=rs2) pc+=im	1100011
BLTU	-	110	If(rs1<rs2) pc+=im	1100011
BGEU	-	111	If(rs1>=rs2) pc+=im	1100011

4.5 U-Type Format

31	12	11	7	6	0
Imm[31:12]		rd		Opcode	

U-Type Instructions

Name	Func7	Func3	Descr	Opcode
LUI	-	-	Rd = imm << 12	0110111
AUIPC	-	-	Rd = pc+(imm<<12)	0010111

4.6 J-Type Format

31	12	11	7	6	0
Imm[20|10:1|11|19:2]		rd		Opcode	

J-Type Instructions

Name	Func7	Func3	Descr	Opcode
JAL	-	-	Rd = pc+4 pc+= imm	1101111
JALR	-	000	Rd=pc+4, pc = rs1 + imm	1100111

5. Control Signal

5.1 Overview

The control logic decodes the instruction (opcode + funct fields) and produces control signals that steer multiplexers, enable writes, select ALU operations, control memory access, and choose the next PC. The Branch Comparator produces BrEq and BrLT, which the control logic uses (together with funct3 and BrUn) to decide whether a branch is taken.

5.2 Table

Signal	Width	Description
PCSel	1	Select next PC
ImmSel	3	Immediate formats select
RegWEn	1	Register writes enable
BrUn	1	Branch compare
ASel	1	ALU input A select
BSel	1	ALU input B select
ALUSel	4	ALU operation select
MemRW	1	Data memory direction
WBSel	2	Writeback selct

5.3 Control signal by Instruction Type

Legend:

- 0= disabled
- 1 = enabled
- X = don't care

Type	PCSel	ImmSel	RegWEn	BrUn	ASel	BSel	ALUSel	MemRWl	WbSel
R	0	xxx	1	0	0	0	decode	0	01
I	0	000	1	0	0	1	decode	0	01
Load	0	000	1	0	0	1	add	0	00
S	0	001	0	0	0	1	add	1	xx
B	depend	010	0	depend	1	1	add	0	xx
JAL	1	100	1	x	1	1	Add	0	10
JALR	1	000	1	X	0	1	Add	0	10
LUI	0	011	1	X	X	1	Pass_b	0	01
AUIPC	0	011	1	x	1	1	add	0	01

6. Hardware Modules

6.1 Program counter

The Program Counter stores the address of the current instruction being executed. At each clock edge, the PC is updated with the next instruction address selected by the PC multiplexer. Normally the PC is incremented by 4 to fetch the next sequential instruction, but it may also be uptaded with a branch or jump target address.

6.2 Register file

The register file contains 32 general purpose register used for operand storage. Two registers can be read simultaneously through the rs1 and rs2 ports, while a third register may be written using rd port when RegWen is selected. Register x0 is hardwired to zero and cannot be modified.

6.3 ALU

The arithmetic logic unit perform aritmetic, logical and shift operations required by the instruction set. The operation executed by the ALU is determined by ALUSel control signal. The operands are selected using the Asel and BSel multiplexer.

6.4 Immediate Generator

The immediate generator extract and sign-extends immediate values from the instruction word according to the instruction format. It is selected using ImmSel control signal.

6.5 Control Unit

The Control Unit decodes the instruction opcode and function field to generate control signal that steer the datapath. These signals control muxes, register writes, ALU operations, memory accesses and the next program counter selection.

6.6 Branch Comparator

The branch comparator evaluates the relationship between rs1 and rs2 to determine branch condition. It generated BrEq and BrLT signals used by the control logic to decided wether a branch should be taken.

6.7 Instruction Memory

The instruction memory stores the program instruction executed by the processor. It is accessed using the address provided by the PC. At each cycle, the instruction located at the address PC is fetched and provided to the datapath for decoding and execution.

In this implementation, instruction memory is read-only during execution and follows the Harvard architecture, meaning that instructions and data are stored in separate memory modules. This avoid structural hazards and allors instruction fetch and data access to occur independently.

6.8 Data Memory

The data memory is used for load and store instructions. The memory address is provided bby the ALU result, which represent the effective address computed from a base register and an immediate offset.

6.9 Muxes

Multiplpexers are used throughput the datapath to select between multiple data sources.

6.10 Adders

Adders are used to compute sequential and branch target addreses. One adder calculates PC +4 .

7. Datapath

The datapath defines how data moves through the processor while executing instructions. The processor follows a single-cycle architecture, meaning that instruction fetch, decode, execution, memory access, and write-back are all completed within a single clock cycle.

At the start of each cycle, the Program Counter (PC) provides a 32-bit address to the Instruction Memory, which returns the corresponding instruction word. At the same time, a dedicated adder computes PC + 4, representing the address of the next sequential instruction.

The fetched instruction is decoded into its constituent fields, including opcode, funct3, funct7, rs1, rs2, rd, and the immediate bits. These fields are used by the Control Unit, which generates the control signals required for the current instruction. The Register File simultaneously reads the contents of registers rs1 and rs2, while also preparing to write a result to register rd if the RegWEn signal is asserted.

For instructions that require an immediate operand, the Immediate Generator extracts and sign-extends the appropriate bits according to the instruction format (I, S, B, U, or J). The specific format is selected using the ImmSel control signal.

The Arithmetic Logic Unit (ALU) performs arithmetic, logical, shift, or comparison operations. Two multiplexers select the ALU operands:

- ASel chooses between the value read from rs1 and the current PC.
- BSel chooses between the value read from rs2 and the generated immediate value.

These selections allow the ALU to perform operations required for arithmetic instructions, effective address calculations, and branch or jump target computation. The exact operation performed by the ALU is determined by the ALUSel control signal.

For memory instructions, the ALU output represents the effective address used by the Data Memory. When MemRW = 1, data from rs2 is written to memory (store instructions). When MemRW = 0, data is read from memory (load instructions).

During the write-back stage, a multiplexer controlled by WBSel selects the value written to the destination register rd. The selected value may be:

- the ALU result (arithmetic and logical instructions),
- the data read from memory (load instructions), or
- PC + 4 (jump instructions such as JAL and JALR).

Branch instructions require a comparison between rs1 and rs2. This comparison is performed by a dedicated Branch Comparator, which generates the signals BrEq (equal) and BrLT (less than). The BrUn control signal determines whether the comparison is signed or unsigned. These signals are evaluated by the Control Unit to determine whether the branch condition is satisfied.

If the branch condition is true, the next PC value is the branch target address, computed by adding the PC and the branch immediate using the ALU. Otherwise, execution continues sequentially with PC + 4.

Jump instructions also modify the normal PC update. For JAL, the next PC is computed as PC + J-type immediate, while JALR computes the target as rs1 + immediate using the ALU. A multiplexer controlled by PCSel selects between the sequential address (PC + 4) and the computed target address. The selected value is written into the PC register at the next clock edge.

Because all stages of execution occur within a single clock period, the clock frequency is limited by the longest combinational path in the datapath. This critical path typically includes instruction fetch, register file access, ALU computation, and data memory access.

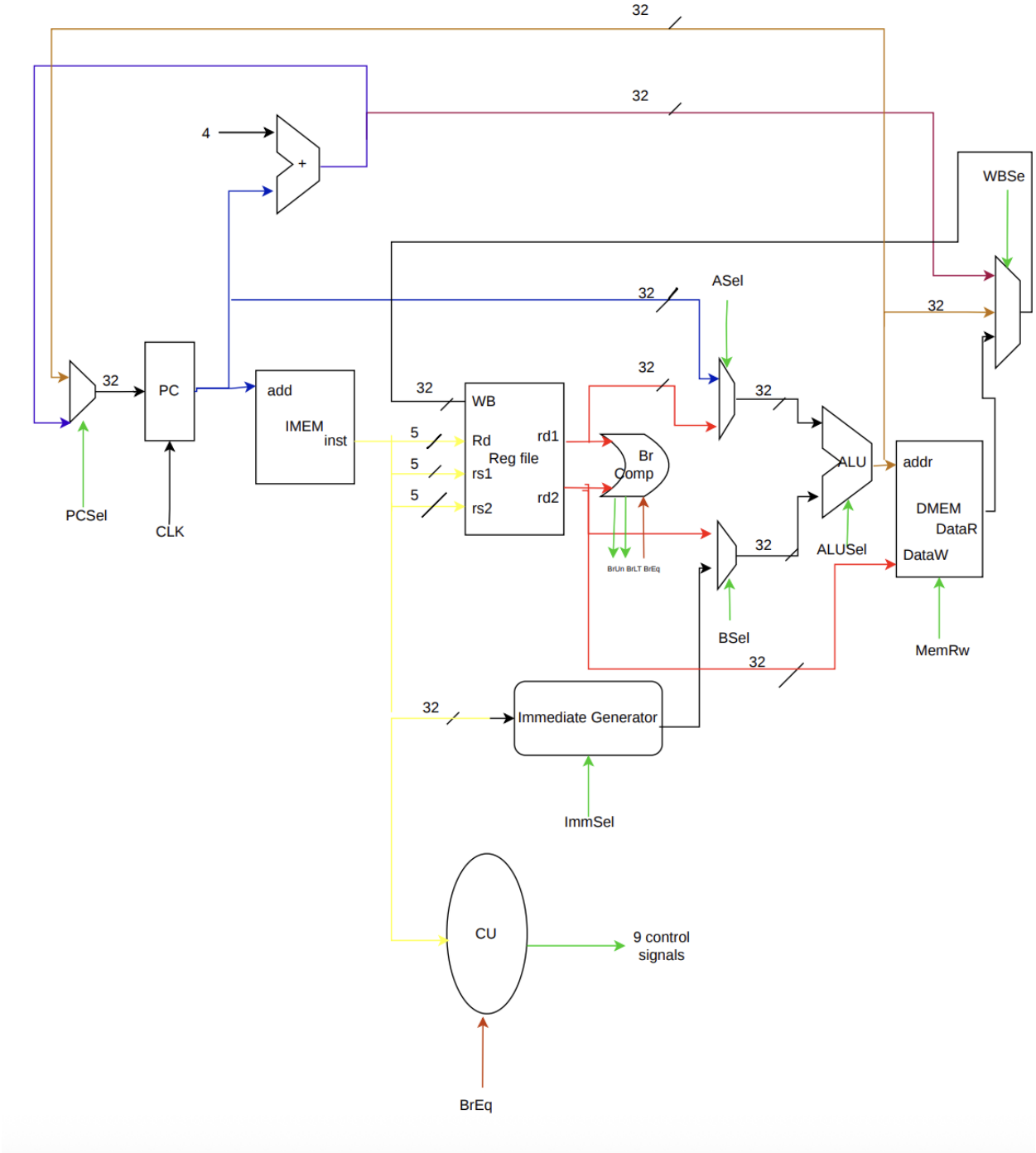


figure 1: Representation of the Datapath

8. Simulation and Results

8.1 ALU

The ALU was verified using a testbench that varied operands a and b while changing the ALUSel control signal. Observed signals included y, zero, lt_signed, and lt_unsigned. The simulation tested ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, and SLTU operations. The waveform confirms correct ALU output selection, proper comparison flags, and correct assertion of zero when the result equals zero..

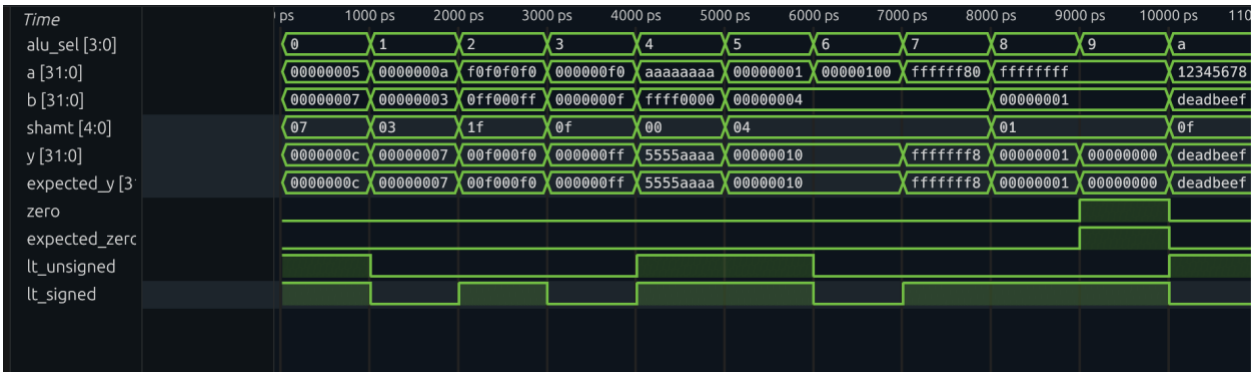


Figure 2: ALU Simulation Waveform

8.2 Register File

A testbench verified the register file by performing write operations on the rising clock edge and reading values through the rs1 and rs2 ports. The waveform confirms correct synchronous writes and simultaneous asynchronous reads.

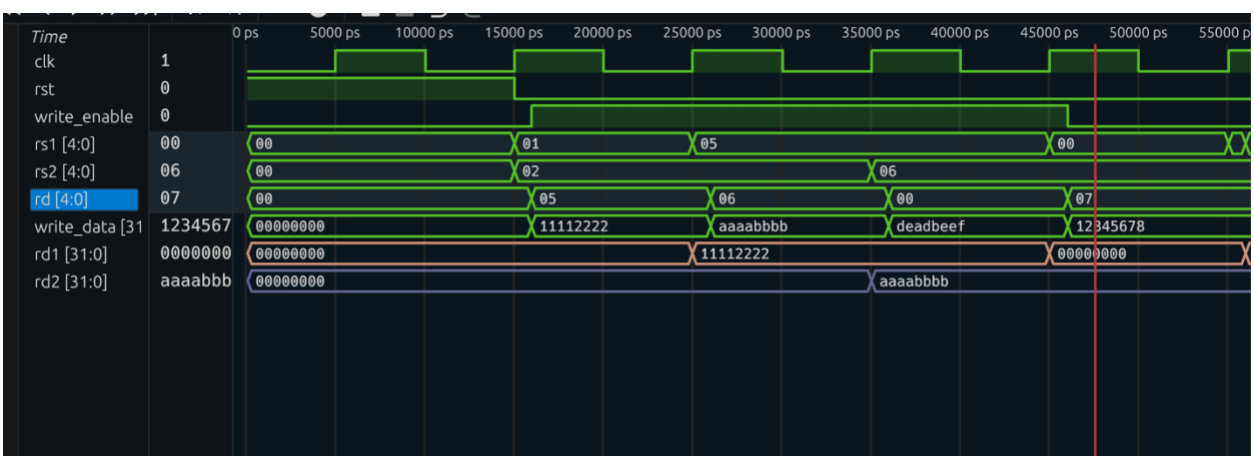


Figure 3: Reg File Simulation Waveform

8.3 Immediate Generator

A testbench verified the Immediate Generator by applying instructions from the I, S, B, U, and J formats. The generated immediate (imm) matches the expected value in all cases, confirming correct extraction and sign extension.

Figure 4: Immediate Gen Simulation Waveform

8.4 Control Unit

A testbench verified the control unit by applying instructions such as ADD, ADDI, LW, SW, and BEQ. The waveform confirms correct decoding of instruction fields and proper generation of control signals including ALUSel, ASel, BSel, ImmSel, PCSel, MemRW, and RegWEn.

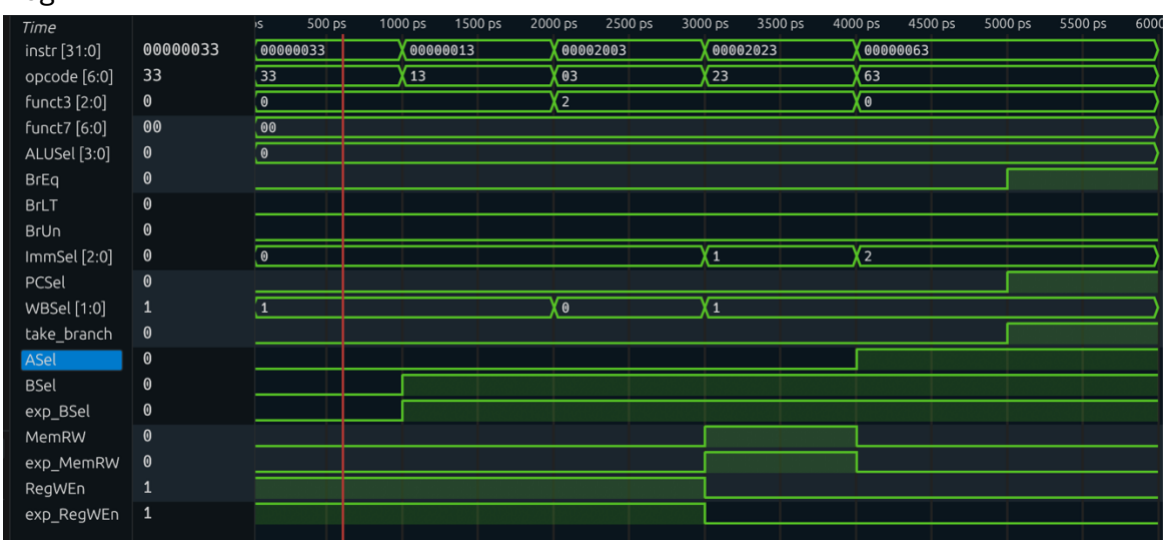


Figure 5: Control Unit Simulation Waveform

8.5 PC

A testbench verified the Program Counter by applying a reset and observing updates on the rising clock edge. The waveform confirms correct initialization to **0x00000000** and proper sequential updates where PC takes the value of next_pc (PC + 4) each cycle.

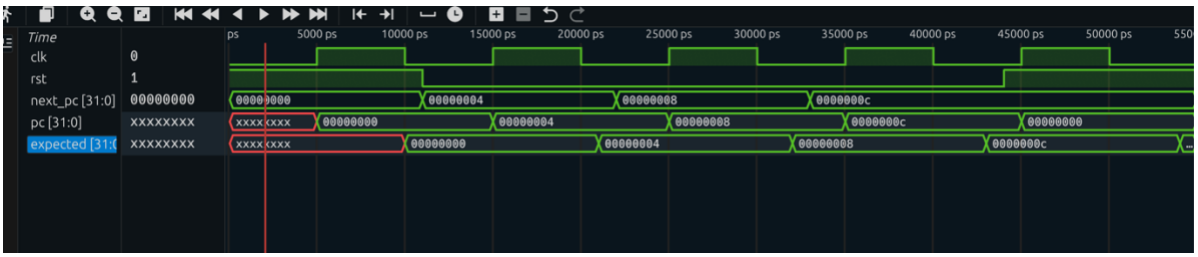


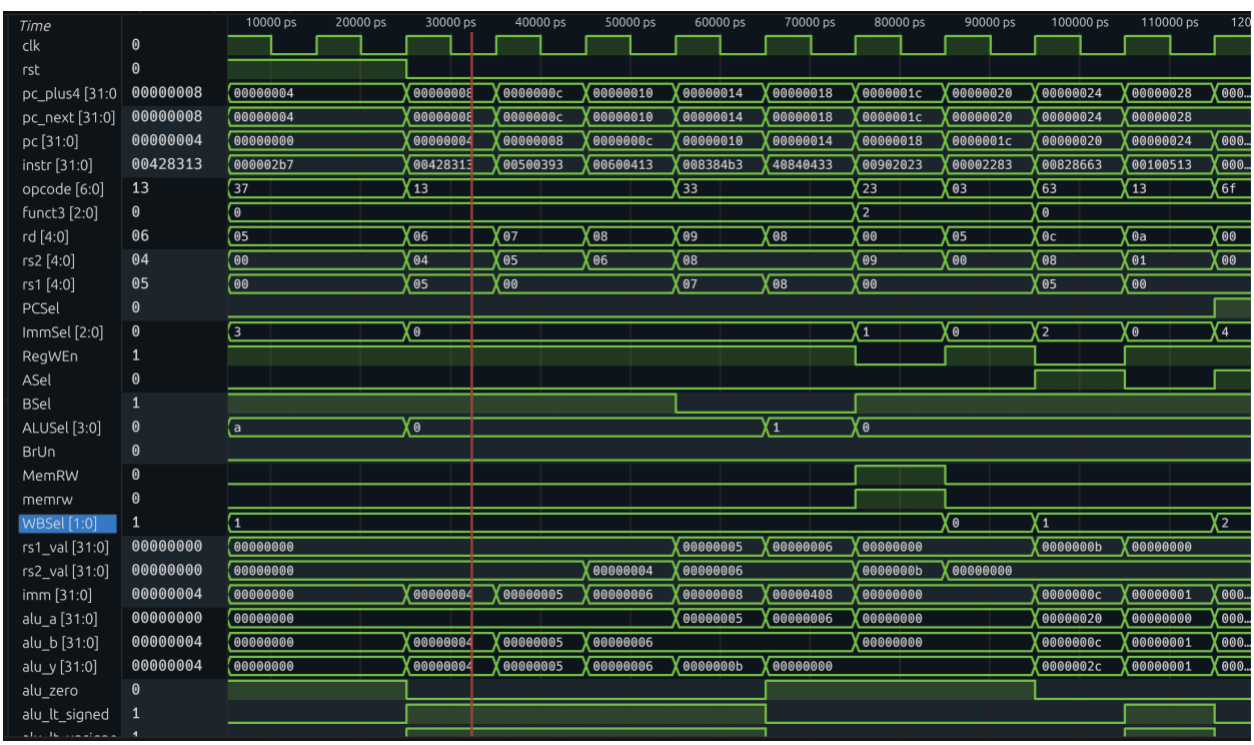
Figure 6: PC Simulation Waveform

8.6 CPU

A testbench verified the complete processor by executing the following RISC-V program loaded in instruction memory:

```
000002B7  → lui   x5,0x0
00428313  → addi  x6,x5,4
00500393  → addi  x7,x0,5
00600413  → addi  x8,x0,6
008384B3  → add   x9,x7,x8
40840433  → sub   x8,x8,x8
00902023  → sw    x9,0(x0)
00002283  → lw    x5,0(x0)
00828663  → beq   x5,x8,label
00100513  → addi  x10,x0,1
0000006F  → jal   x0,label
00200513  → addi  x10,x0,2
004000EF  → jal   x1,4
0000006F  → jal   x0,label
00000013  → nop
00008067  → jalr  x0,x1,0
00000013  → nop
```

The waveform shows correct PC updates, instruction fetch from instruction memory, ALU execution for arithmetic operations, proper load/store memory accesses, and correct branch and jump behavior. This confirms correct integration and interaction of all major datapath components.



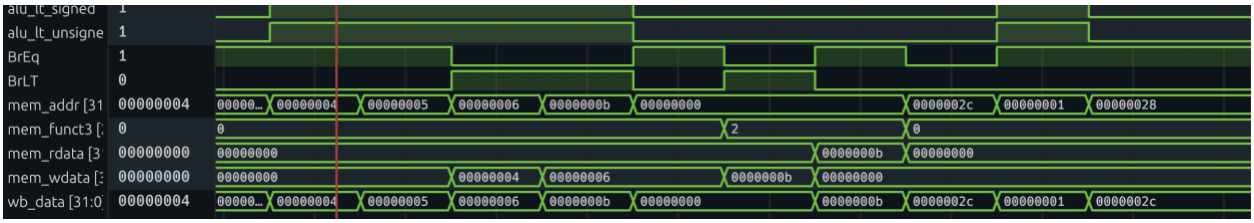


Figure 7: CPU Simulation Waveform

9.Limitation of design

Although the design functions correctly, the single-cycle architecture introduces several limitations related to performance and hardware utilization. Every instruction must complete within one clock period. As a result, the clock duration must be long enough to accommodate the longest execution path, which limits the maximum achievable frequency. Additionally, all functional units including the ALU, memory blocks, and adders are active in every cycle, even when they are not required for a particular instruction. This leads to inefficient hardware utilization, as certain components such as multiplexers remain idle depending on the instruction type.

10.Summary

This project implements a fully functional 32-bit RV32I single-cycle RISC-V processor in Verilog. The processor supports the major RISC-V instruction formats including R-type, I-type, S-type, B-type, U-type, and J-type instructions.

The CPU architecture consists of a modular Datapath, a hardwired control unit, and interfaces to instruction and data memory. The design includes all essential processor components such as the Program Counter, Register File, Arithmetic Logic Unit (ALU), Immediate Generator, Control Unit, and memory access logic. Each module was independently verified through simulation and then integrated into the complete processor system.

Simulation results confirm that the processor correctly executes instructions, performs arithmetic and logical operations, accesses memory, and handles control flow instructions such as branches and jumps.

11.Future work

While the current design successfully implements a functional single-cycle RV32I processor, several improvements can be made to enhance performance and extend the architecture.

The next step in improving this processor would be to implement a pipelined architecture. By dividing instruction execution into multiple stages such as Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB), the processor will be able to process multiple instructions simultaneously, significantly improving performance compared to the current single-cycle design.